

SIMATS ENGINEERING

ASSESSMENT TOOL 3 – MOCK INTERVIEW

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

CO5: Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.

Name: VAISHNAV M

Register No.: 192524251

MOCK INTERVIEW – ANSWERS

Q1. What is the primary goal of code generation, and how does it affect program performance?

Answer: Code generation converts intermediate representation into target-machine code. Its main goal is to produce correct and efficient code while minimizing execution time, memory use, and code size. Better code generation generally improves program performance.

Q2. How do target machine characteristics influence code generation?

Answer: Target-machine features such as instruction set, number of registers, register types, addressing modes, memory organization, and instruction costs directly influence instruction selection, register allocation, scheduling, and the final quality of generated code.

Q3. Explain the role of intermediate code in optimization and portability.

Answer: Intermediate code provides a machine-independent representation between source code and target code. Optimizations can be performed on this representation without depending on a particular processor, improving portability and allowing one front end to support multiple target machines.

Q4. Why is machine-independent optimization preferred before machine-dependent optimization?

Answer: Machine-independent optimization can improve the program before target-specific details are introduced. This makes the optimizations reusable across processors and provides a cleaner basis for later machine-dependent improvements such as register allocation and instruction selection.

Q5. What are the major sources of inefficiency in unoptimized code?

Answer: Major inefficiencies include redundant computations, unnecessary assignments, dead code, excessive memory accesses, poor register use, unnecessary jumps, weak instruction selection, and inefficient loops.

Q6. How does common subexpression elimination reduce computation time?

Answer: Common subexpression elimination (CSE) identifies expressions computed more than once and reuses an earlier result when the operands have not changed. For example, if `a*b` is computed twice without changes to `a` or `b`, the second computation can reuse the first result, reducing execution time.

Q7. Can common subexpression elimination ever be unsafe? Justify your answer.

Answer: Yes. CSE is unsafe when evaluating the expression again could have a different result or side effect. Changes to operands, function calls with side effects, volatile memory, or aliasing can invalidate the earlier result. The compiler must prove that reuse preserves program semantics.

Q8. Explain how constant folding and constant propagation work together.

Answer: Constant folding evaluates constant expressions at compile time, such as `3*4` becoming `12`. Constant propagation replaces a variable with a known constant value, such as replacing `x` with `10` after `x=10`. Together they expose more compile-time computations and enable further optimization.

Q9. How do you detect and eliminate dead code in a program?

Answer: Dead code is code whose result can never affect observable program behavior. Compilers use control-flow and data-flow information, such as liveness analysis, to identify unused assignments and unreachable statements, then remove them safely.

Q10. What are the limitations of peephole optimization?

Answer: Peephole optimization examines a small sequence of target instructions at a time. Its limitations include a local view, dependence on target architecture, difficulty detecting global opportunities, and inability to replace some higher-level algorithmic inefficiencies.

Q11. Explain different peephole optimization techniques with examples.

Answer: Common techniques include removing redundant instructions, eliminating unnecessary loads/stores, simplifying jumps, replacing expensive instructions with cheaper ones, and combining instruction sequences. Example: `ADD R1, 0` can be removed because it does not change R1; an unconditional jump to the immediately following instruction can also be removed.

Q12. How does strength reduction improve execution efficiency?

Answer: Strength reduction replaces an expensive operation with a cheaper equivalent operation. For example, repeatedly computing `i*4` inside a loop can be replaced by maintaining a value that is increased by 4 each iteration. This can reduce multiplication cost and improve loop performance.

Q13. What is the role of algebraic simplification in optimization?

Answer: Algebraic simplification applies identities such as `x+0=x`, `x*1=x`, and `x*0=0` where they are semantically valid. It removes unnecessary operations and can expose additional optimization opportunities.

Q14. How are redundant instructions identified and removed?

Answer: Redundant instructions are identified using control-flow and data-flow information, together with local instruction analysis. If an instruction has no observable effect or repeats a

computation whose result is already available, it can be removed or replaced while preserving program behavior.

Q15. Why are basic blocks important in optimization?

Answer: Basic blocks are straight-line sequences with one entry and one exit. They provide manageable units for local optimization, data-flow analysis, DAG construction, and control-flow graph construction.

Q16. What conditions must a sequence of statements satisfy to form a basic block?

Answer: A basic block has one entry point and one exit, with no branching into the middle of the block and no branching out except at its end. Leaders are identified as the first statement, branch targets, and statements immediately following branches; statements from each leader to before the next leader form a block.

Q17. Explain the process of constructing a control flow graph (CFG).

Answer: To construct a CFG, first divide the program into basic blocks. Create one node for each block, then add a directed edge from block A to block B whenever control can transfer from A to B, for example through a conditional branch, unconditional jump, or fall-through.

Q18. How does a CFG help in program analysis and optimization?

Answer: A CFG represents possible execution paths through a program. It supports reachability, liveness, reaching definitions, loop detection, dead-code elimination, branch analysis, and many other global optimizations.

Q19. Compare local optimization vs global optimization with examples.

Answer: Local optimization operates within a single basic block, such as eliminating a repeated expression in that block. Global optimization considers multiple basic blocks along control-flow paths, such as global common subexpression elimination or reaching-definitions-based transformations.

Q20. What are the challenges in global data flow analysis?

Answer: Global data-flow analysis must handle branches, loops, multiple definitions, paths that merge, and iterative computation until information reaches a fixed point. It must also remain efficient and conservative so that transformations do not change program meaning.

Q21. How does live variable analysis help in optimization?

Answer: Live variable analysis determines whether a variable's current value may be used in the future before being overwritten. If a variable is not live after an assignment, that assignment may be removable. It also helps register allocation by identifying values that need to remain available.

Q22. Explain the concept of reaching definitions with a practical example.

Answer: Reaching definitions analysis determines which assignments may reach a program point without being overwritten. For example, after `x=5`, if no later assignment to `x` occurs on a path, that definition reaches a use of `x`. It helps with constant propagation, use-definition relationships, and other optimizations.

Q23. What is available expression analysis, and where is it used?

Answer: Available expression analysis finds expressions that have already been computed on every path to a program point and whose operands have not changed. It is useful for global common subexpression elimination and reducing repeated computations.

Q24. How do data dependencies affect instruction execution?

Answer: Data dependencies describe ordering constraints between instructions. A true dependency occurs when one instruction needs a value produced by another. Anti- and output dependencies can also constrain reordering. Correct scheduling must preserve these dependencies.

Q25. What are control dependencies, and why are they important?

Answer: Control dependency means whether execution of one statement depends on a control decision such as an 'if' or loop condition. It is important for preserving program behavior when moving, eliminating, or speculating instructions.

Q26. What are the key design issues in a code generator?

Answer: Key design issues include correctness, instruction selection, register allocation, evaluation order, instruction scheduling, addressing modes, target-machine constraints, memory management, and balancing execution speed against code size.

Q27. How is instruction selection performed in compilers?

Answer: Instruction selection maps intermediate operations to target instructions. The compiler considers available instructions, operand forms, addressing modes, instruction costs, and register constraints, often using pattern matching or tree/DAG-based techniques to choose efficient sequences.

Q28. What factors affect register allocation strategies?

Answer: Register allocation depends on the number and types of registers, variable lifetimes, interference between live values, calling conventions, instruction constraints, and the cost of spilling values to memory.

Q29. When does register spilling occur, and how is it handled?

Answer: Register spilling occurs when there are more simultaneously needed live values than available registers. The compiler chooses some values to store in memory temporarily and reloads them later. Good spilling decisions minimize extra loads and stores.

Q30. How does next-use information guide register allocation?

Answer: Next-use information indicates where a variable will be used again. When a register is needed, the compiler can prefer a value that has no future use or whose next use is farthest away, reducing unnecessary spills and reloads.

Q31. What is the role of a simple code generator?

Answer: A simple code generator translates intermediate code, often three-address statements, into target instructions using a small set of rules. It demonstrates fundamental code-generation tasks such as choosing instructions, managing registers, and storing values.

Q32. What assumptions are made in designing a simple code generator?

Answer: A simple code generator commonly assumes a fixed target instruction set, a limited register set, simple addressing modes, sequential execution, and a straightforward intermediate representation. It usually ignores advanced optimizations and complex architectural features.

Q33. How are register descriptors and address descriptors used?

Answer: A register descriptor records which variables or temporary values are currently held in each register. An address descriptor records where the current value of a variable may be found, such as a register or memory location. Together they help avoid unnecessary moves and stores.

Q34. Explain the structure and purpose of a target machine.

Answer: A target machine is the processor and execution environment for which code is generated. Its structure includes registers, arithmetic/logic units, memory, instruction set, addressing modes, and calling conventions. These characteristics determine many code-generation decisions.

Q35. Compare register-based and stack-based machines in code generation.

Answer: Register-based machines perform operations mainly on named registers, usually reducing memory traffic and enabling efficient execution. Stack-based machines operate on values at the top of a stack, simplifying some instruction formats but often requiring more stack operations. Register machines are common in modern general-purpose processors.

Q36. What are different addressing modes, and how do they affect code generation?

Answer: Addressing modes specify how an instruction obtains an operand, such as immediate, register, direct, indirect, indexed, or base-plus-offset addressing. They influence instruction count, operand access cost, and how efficiently arrays, structures, and pointers can be handled.

Q37. How does instruction scheduling improve performance?

Answer: Instruction scheduling reorders independent instructions to reduce pipeline stalls and keep functional units busy. It must preserve data and control dependencies while considering latency, throughput, and target-processor constraints.

Q38. What is runtime storage management, and why is it necessary?

Answer: Runtime storage management controls allocation and release of memory while a program executes. It is necessary for activation records, dynamic data structures, objects, recursion, and dynamically allocated memory.

Q39. Compare stack allocation and heap allocation.

Answer: Stack allocation manages storage in last-in-first-out order and is efficient for procedure calls and local variables. Heap allocation supports dynamically sized and long-lived objects but generally requires more complex management and can have higher overhead.

Q40. What is an activation record, and what are its components?

Answer: An activation record, or stack frame, stores information for a procedure invocation. Typical components include parameters, return address, saved registers, local variables, temporaries, control/access links where applicable, and bookkeeping information.

Q41. How is a DAG constructed for a basic block?

Answer: A DAG for a basic block is built by creating leaf nodes for initial values and operator nodes for expressions. Each node records an operator and its operand nodes. If the same expression with unchanged operands occurs again, the existing node can be reused rather than creating a duplicate.

Q42. How does a DAG help in common subexpression elimination?

Answer: DAG-based CSE identifies identical operator nodes with identical children. Multiple computations can then refer to the same node, so the compiler generates the expression once and reuses its value.

Q43. How does DAG representation assist in dead code elimination?

Answer: A DAG shows which computed values are needed by later statements. Nodes whose values are never used and have no required side effects can be omitted, supporting dead-code elimination within a basic block.

Q44. What are the advantages of using DAG over syntax trees?

Answer: DAGs explicitly represent value dependencies and shared subexpressions, whereas ordinary syntax trees may duplicate identical subtrees. DAGs therefore make common subexpression detection and local value reuse more direct and efficient.

Q45. What is loop optimization, and why is it important?

Answer: Loop optimization improves frequently executed loops to reduce total execution time. Common techniques include loop-invariant code motion, strength reduction, induction-variable elimination, loop unrolling, and fusion where applicable.

Q46. Explain loop-invariant code motion with an example.

Answer: Loop-invariant code motion moves a computation outside a loop when its value does not change across iterations and moving it preserves program semantics. Example: if `'x = a*b'` where `'a'` and `'b'` are unchanged in the loop, compute `'x'` once before the loop instead of every iteration.

Q47. What is induction variable elimination, and how does it help?

Answer: An induction variable changes by a fixed amount on each loop iteration, such as `'i=i+1'`. Induction-variable elimination removes unnecessary induction variables or derives one from another, reducing instructions and simplifying loops.

Q48. How does loop unrolling improve performance?

Answer: Loop unrolling replicates the loop body so several iterations execute per loop-control operation. It can reduce branch and counter overhead and expose more instruction-level parallelism, although it may increase code size.

Q49. What is code motion across basic blocks?

Answer: Code motion across basic blocks moves a computation from one block to another when data-flow and control-flow conditions guarantee that the moved computation is executed correctly and has the same observable effect. It is often used to move loop-invariant computations out of loops.

Q50. How do modern compilers combine multiple optimization techniques effectively?

Answer: Modern compilers apply multiple passes iteratively. They may perform constant folding, propagation, CSE, dead-code elimination, loop optimizations, instruction selection, register allocation, and scheduling, with one optimization exposing opportunities for another. The process is guided by correctness, target-machine characteristics, and performance goals.